

STAT 432 Fall 2026

Homework 5

Eric Chen
September 27, 2026

Problem 1

In our previous homework, we used repeated simulations to study the behavior of linear regression estimators. We will now use the same idea for KNN. Consider the regression model

$$Y_i = f(\mathbf{X}_i) + \varepsilon_i, \quad f(\mathbf{x}) = 0.5x_1 + \sin(x_2) - 0.3x_3^2,$$

where the three covariates are independent standard normal variables and $\varepsilon_i \sim \mathcal{N}(0, 0.1^2)$ independently of the covariates and other observations. Generate $n = 400$ training observations. Our goal is to estimate the mean response at

$$\mathbf{x}_0 = (0.5, 0.7, 1)^T.$$

Use Euclidean distance and consider $k = 1, 3, 5, \dots, 29$.

- Calculate $f(\mathbf{x}_0)$. Generate one training dataset and obtain the KNN prediction at $\mathbf{x}_0$ for each value of k .
- Independently repeat the generation of the training covariates and responses 200 times. Within each repetition, use the same training data for every value of k . Store the predictions in a matrix with 200 rows and one column for each value of k . Use this matrix to estimate the bias, variance, and mean squared error of the estimated mean response. Use divisor 200 when calculating the empirical variance, and verify numerically that

$$\text{mean squared error} = \text{squared bias} + \text{variance}.$$

Plot squared bias, variance, and mean squared error against k in one figure. Explain the pattern and compare it with the bias-variance trade-off for ridge regression or lasso.

- $f(\mathbf{x}_0) = 0.5 \cdot 0.5 + \sin(0.7) - 0.3 = 0.594$.

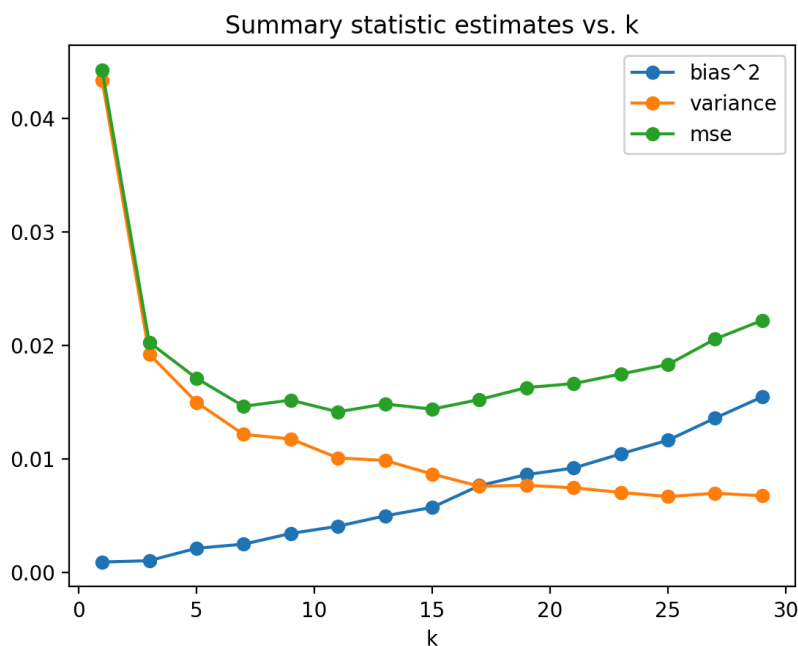
We get the following table for the KNN prediction at $\mathbf{x}_0$ for all k :

k	$\hat{x}_0$ prediction
1	0.886111
3	0.780870
5	0.614012
7	0.529068
9	0.557340
11	0.598498
13	0.623486
15	0.569935
17	0.522069
19	0.534670
21	0.566064
23	0.539307
25	0.494147
27	0.494316
29	0.520445

b. We get the following table for the summary statistics

k	bias	variance	mse	bias^2	$\text{bias}^2 + \text{variance}$
1	-0.030036	0.043352	0.044255	0.000902	0.044255
3	-0.032048	0.019232	0.020259	0.001027	0.020259
5	-0.046025	0.014979	0.017097	0.002118	0.017097
7	-0.049777	0.012150	0.014628	0.002478	0.014628
9	-0.058487	0.011749	0.015169	0.003421	0.015169
11	-0.063704	0.010078	0.014136	0.004058	0.014136
13	-0.070518	0.009857	0.014830	0.004973	0.014830
15	-0.075634	0.008654	0.014374	0.005720	0.014374
17	-0.087371	0.007585	0.015219	0.007634	0.015219
19	-0.092775	0.007667	0.016274	0.008607	0.016274
21	-0.095844	0.007439	0.016626	0.009186	0.016626
23	-0.102150	0.007036	0.017470	0.010435	0.017470
25	-0.107872	0.006666	0.018302	0.011636	0.018302
27	-0.116555	0.006966	0.020551	0.013585	0.020551
29	-0.124331	0.006738	0.022196	0.015458	0.022196

and the following graph:



We observe similar results to the bias-variance tradeoff for regression. Increasing k increases the bias (since our model no longer describes the patterns in the data as accurately) while our variance decreases (for small k , we overfit on the data.) The ultimate result is another U-shaped curve, where on the left, the large variance leads to a bad model, and on the right, the large bias leads to a bad model.

Problem 2

Use the response model from Question 1, including the noise standard deviation 0.1, but generate $p = 30$ covariates. The response still depends only on the first three. Consider these two settings:

- **Setting 1:** all 30 covariates are independent standard normal variables.
- **Setting 2:** the covariates have a multivariate normal distribution with mean zero and covariance matrix Σ , where

$$\Sigma_{jj} = 1, \quad \Sigma_{jl} = 0.8 \quad \text{for } j \neq l.$$

In each setting, generate 400 training observations and 1,000 independent test observations. Fit 5NN using Euclidean distance on the supplied covariate scale. All covariates already have population variance one.

For lasso, use a linear model containing the 30 covariates and a separately fitted, unpenalized intercept. Follow the Week 4 objective,

$$L_\lambda(\beta) = \frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1,$$

where $\mathbf{X} \in \mathbb{R}^{n \times 30}$ contains the centered and standardized covariates and $\tilde{\mathbf{y}}$ is the centered training response. Select the penalty by ten-fold cross-validation over 41 logarithmically spaced values from 10^{-4} to 1. Estimate all means and scales using only the training portion of each fold, with divisor equal to that portion's sample size. After selecting the penalty,

refit on all training observations and apply the fitted transformations unchanged to the test observations. Do not add nonlinear terms to the lasso model.

- Fit both methods in each setting. Use the same training and test data for the two methods within a setting. Report their test MSE values in one table, together with the selected lasso penalty and the number of nonzero lasso slopes. You only need to generate one dataset for each setting; repeated simulations are not required.
- Compare the results between the two settings. Explain how correlation can change the usefulness of Euclidean neighborhoods, and why lasso does not use all covariates in the same way as KNN.
- The response model contains $\sin(X_2)$ and X_3^2 , but the lasso fit uses only linear terms. Explain how this difference affects the comparison. Would the results justify a general claim that correlation always helps KNN, or that one method is always better? Explain why.

a.

Design	5NN test MSE	Lasso test MSE	Selected λ	Nonzero slopes
Independent	0.5548	0.2698	0.03162	7
Correlated ($\rho = 0.8$)	0.1770	0.3103	0.01259	15

- We observe that correlation improves the 5NN test MSE compared to independently distributed data. This makes sense, since we want our data to form clusters for better prediction. When the covariates are randomly distributed, the five nearest neighbors may be quite far away, and thus give a poor estimate. Lasso does not use as many covariates, since the penalty leads to some of the covariates being shrunk to zero.
- The lasso model is still somewhat reasonable for X_2 , since $\sin(X_2)$ is roughly linear for $X_2 \in [-1, 1]$. However, since X_3^2 clearly does not follow a linear trend for $X_3 \in [-1, 1]$, we expect it to be more biased in that situation. As a result, we expect KNN to perform better here. However, the results show that there is no clear better model. In the case where the variables are independently distributed, Lasso performed better, and in the case that the variables are more highly correlated, KNN performed better.

This makes sense, since the advantage KNN gets for being less biased for X_3^2 is not significant enough to outweigh the downside it gets for sparse data. but becomes enough to perform better than Lasso for clustered data.

Problem 3

The lecture's handwritten digit example shows that KNN can work with many measured covariates. We will use a simulation to investigate why the structure of those covariates matters.

Let $p = 100$ be the number of observed covariates and let m be the number of latent variables. For each observation, generate

$$\mathbf{X}_i = \mathbf{A}\mathbf{Z}_i + \boldsymbol{\eta}_i, \quad Y_i = Z_{i1} + Z_{i2} + \varepsilon_i,$$

where $\mathbf{A} \in \mathbb{R}^{100 \times m}$ has independent entries from $\text{Uniform}[-2, 2]$. The entries of $\mathbf{Z}_i \in \mathbb{R}^m$ and $\boldsymbol{\eta}_i \in \mathbb{R}^{100}$, and the scalar ε_i , are independent standard normal variables, also independent of $\mathbf{A}$. Generate $\mathbf{A}$ once and use it for all training and test observations in that dataset.

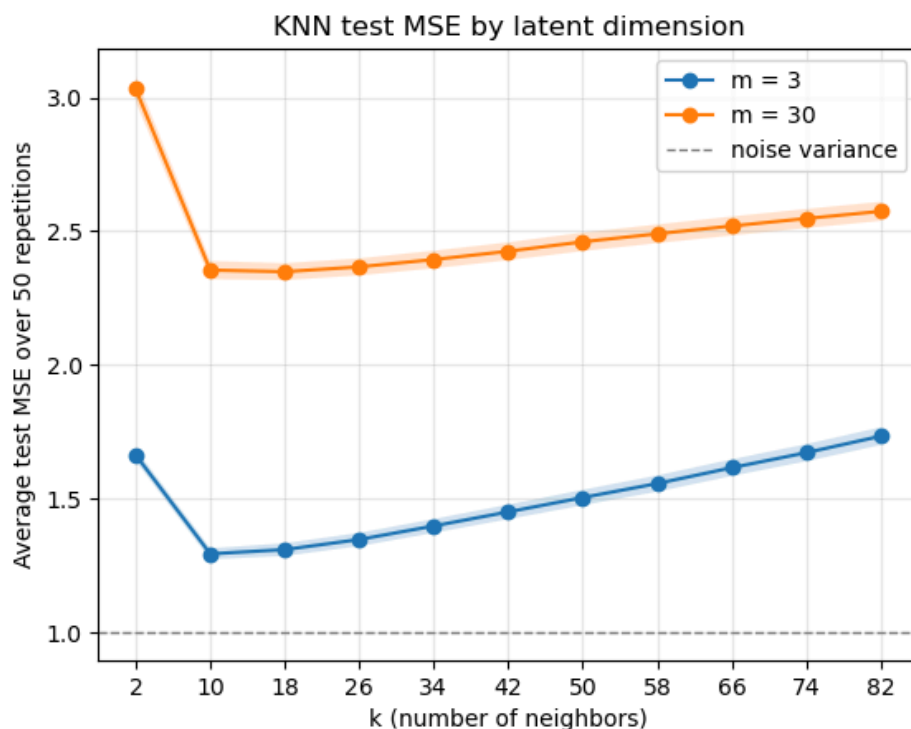
Compare $m = 3$ and $m = 30$. In each setting, generate 200 training observations and 200 independent test observations. Fit KNN using only the 100 observed covariates in $\mathbf{X}$, with Euclidean distance on their generated scale. The latent variables in $\mathbf{Z}$ are used to generate the data but are not available to the fitted model. Consider $k = 2, 10, 18, \dots, 82$.

- Generate one dataset for each value of m . Fit KNN for every value of k and calculate test MSE. Within a setting, use the same training and test observations for all values of k .
- Repeat the complete experiment 50 times, generating a new $\mathbf{A}$ and new observations in each repetition. Plot average test MSE against k , with one curve for each value of m . These curves compare the specified choices of k in a controlled simulation; they are not a cross-validation procedure for selecting a final model.
- Both settings have 100 observed covariates, and the response depends on only two latent variables. Explain why their KNN performance can still differ. What role does the noise η_i play, and why are the observed data not confined exactly to an m -dimensional linear space?

a.

k	m = 3	m = 30
2	1.607918	2.744826
10	1.256992	2.377459
18	1.282546	2.338385
26	1.340470	2.360125
34	1.336491	2.321141
42	1.392700	2.385609
50	1.454743	2.408691
58	1.487196	2.444888
66	1.550414	2.469684
74	1.617419	2.484347
82	1.676908	2.500779

b.



- c. The points in the m -dimensional space will be sparser for larger m , if there is a fixed number of total points. So we expect the test MSE to be larger when m is larger. The η terms increase the bias of the model, since the covariate fed into the model is not the actual Z that generates the Y .

Problem 4

Use `zip.train` and `zip.test` from the R package `ElemStatLearn`, as in the lecture. The package has been retired from the active CRAN repository. If needed, install the archived package into your usual R library, then load the two datasets. Python users can use the course copies of the same training data and test data. These compressed CSV files have no headers and are included in the homework ZIP under `data/knn`. Run the Python code from the folder containing `data`.

Each image has $16 \times 16 = 256$ pixel values. The first column is the digit label and the remaining columns are the pixel values. Use the first 1,000 rows of `zip.train` and the first 500 rows of `zip.test`. Use the original pixel values without standardization, and do not include the digit label in a distance calculation.

For a test image $\mathbf{x}_0$ and a training image $\mathbf{x}_i$, the two distances are

$$d_E(\mathbf{x}_0, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{256} (x_{0j} - x_{ij})^2}, \quad d_M(\mathbf{x}_0, \mathbf{x}_i) = \sum_{j=1}^{256} |x_{0j} - x_{ij}|.$$

- a. Write a 5NN classification function that allows either Euclidean or Manhattan distance. For each test image, calculate its distances to the training images directly, find the five nearest, and return the most frequent digit label. Break distance ties by the smaller training row number and voting ties by the smallest digit. Do not use a built-in distance or KNN function for this part.

- b. Apply the classifier to all 500 test images using each distance. For each fit, report the test classification error and a 10×10 confusion matrix, with rows representing predicted digits and columns representing true digits. Keep all digit labels from 0 through 9 in the matrix, including any with a zero count. Which digits are most often confused with one another?
- c. For Euclidean distance, display three incorrectly classified test images, together with their true and predicted labels. Comment on whether the mistakes are visually understandable. Compare the overall results of the two distance measures. Can they have similar total error while making mistakes on different images? Use your predictions to support your answer.

a. In code

b.

Distance	Test Error	# Misclassified
Euclidean	0.122	61
Manhattan	0.138	69

. Confusion matrix:

true \ predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

. Most confused:

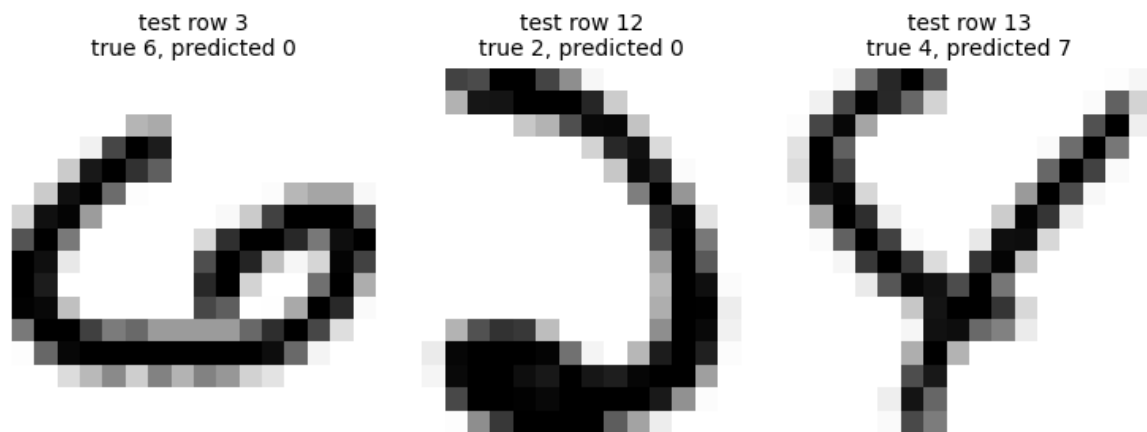
	Euclid.					Manha				
	digit a	digit b	total	pred a / true b	pred b / true a	digit a	digit b	total	pred a / true b	pred b / true a
0	0	2	7	7	0	7	9	6	5	1
1	4	9	5	1	4	0	2	6	6	0
2	7	9	5	4	1	0	5	5	5	0
3	3	5	5	4	1	4	9	5	1	4
4	0	5	4	4	0	3	5	5	4	1

.

Errors. Euclidean 5NN misclassifies 61 of 500 test images (test error 0.122); Manhattan 5NN misclassifies 69 (test error 0.138).

Most confused digits. Rows are predicted and columns are true digits. With Euclidean distance the largest off-diagonal cell is true 2 predicted as 0 (7 images), and the most confused pairs overall are 0/2 (7), then 4/9, 7/9 and 3/5 (5 each) and 0/5 (4). Manhattan shows the same pattern: 0/2 and 0/8 (6 each), then 0/5, 4/9 and 3/5 (5 each). The recurring pairs are digits with similar strokes: a 2 with a closed bottom loop resembles a 0, 4 and 9 share an upper loop or triangle over a vertical stroke, 7 and 9 share a top bar with a descending stroke, and 3 and 5 share the same lower curve. Both classifiers also over-predict 0 (row 0 sums to 134 and 137 versus 119 true zeros), because 0 is the most common training digit and has a large, heavy shape that many poorly written digits land near.

c.



In my opinion, all three are understandable. On the left, the top part and the right swirl of the 6 are close to touching, making it look like a zero. On the middle, it appears similar to a zero where the left edge has been cut-off. On the right, it is a little bit less understandable, but it looks somewhat like a seven with the top line crossing the long bar.

Overall results:

	case	count
0	both wrong	58
1	...same wrong label	50
2	...different wrong labels	8
3	only Euclidean wrong	3
4	only Manhattan wrong	11
5	both correct	428

Generally, both models were able to identify the same images. In 478/500 of the cases, both of the models picked the same label. Both performed similarly, with Euclidean having test error 0.122 and Manhattan having test error 0.138.

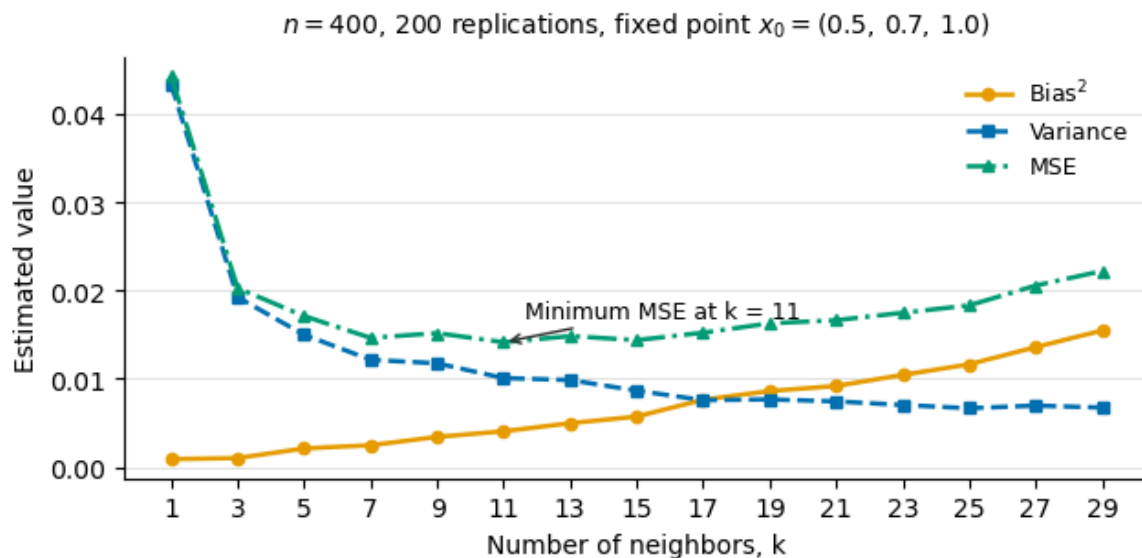
Problem 5

Create a short `SKILL.md` file for improving the formatting of STAT 432 homework plots. You may use AI to help write it.

- Include a title, a description, and a few formatting rules. The description should limit the skill to STAT 432 homework plot improvements and require an explicit request to use it. Your rules should address margins, titles, axes, labels, colors, and sizes for plots produced in R or Python. Keep them portable: do not refer to specific homework questions.
- Ask your agent to install the skill. Explicitly call it to improve the plot from Question 1, then compare the original and revised versions. Do you like the changes? You may revise the skill and test it a couple more times, or uninstall it if it remains unhelpful.
- Append your final `SKILL.md` content to your submission and briefly explain your choices and experience.

- Placed in the folder
- This is the new plot created using my first iteration of `SKILL.md`:

Bias-variance summary across k



I do not really like the formatting of it, as there appears to be a big gap

- Located in the github submission.